

智能计算，也称计算智能，通常指直接模仿自然界智能行为的一类具体算法模型的集合。

- 向大自然学习

- 进化计算，如遗传算法(GA)

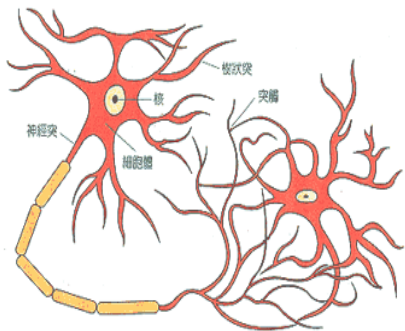
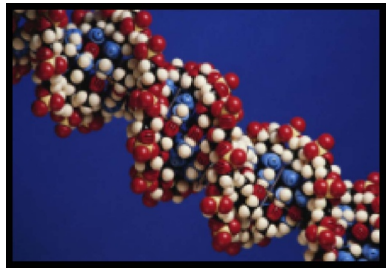
- 模仿生物进化过程：物竞天择，设计染色体编码，根据适应度函数进行染色体选择、交叉和变异操作，优化求解

- 人工神经网络(ANN)

- 模仿大脑神经元，透过神经元的信息传递、训练学习、联想，优化求解

- 模糊逻辑 (Fuzzy Logic)

- 模仿人类思维的模糊性



- 6.1 进化算法的产生与发展
- 6.2 遗传算法及其应用
- 6.3 粒子群优化算法及其应用
- 6.4 蚁群算法及其应用(自学)

- 掌握**遗传算法**的基本概念和基本方法，了解一些遗传算法的改进算法，掌握遗传算法的应用；
- 掌握**粒子群优化算法**等群智能算法的原理、步骤及应用，理解群智能算法与遗传算法的区别。



- **进化计算(Evolutionary Computation, EC)**是20世纪60年代开始发展、90 年代兴起的一门**模拟生物进化与遗传规律**的计算学科。已发展成为与**人工神经网络、模糊逻辑**相并列的智能计算支撑技术。
- **遗传算法(Genetic algorithms, GA)**: 二十世纪六十年代初, 美国Michigan大学的J. H. Holland 教授提出, 强调**染色体的操作**。
- **进化策略(Evolution strategies, ES)**: 1965年, 由Rechenberg和Schwefel独立提出, 强调**个体级的行为变化**。
- **进化规划(Evolutionary programming, EP)**: 由Fogel在20世纪60年代提出, 强调**种群级上的行为变化**。
- **遗传编程 (Genetic Programming, GP)**: 1992年, Stanford 大学的J.R.Koza提出, 采用**层式编码结构 (树形结构)**。
- **基因表达式编程(Gene Expression Programming , GEP)**: 2001 年, 葡萄牙人Candida Ferreira 提出。

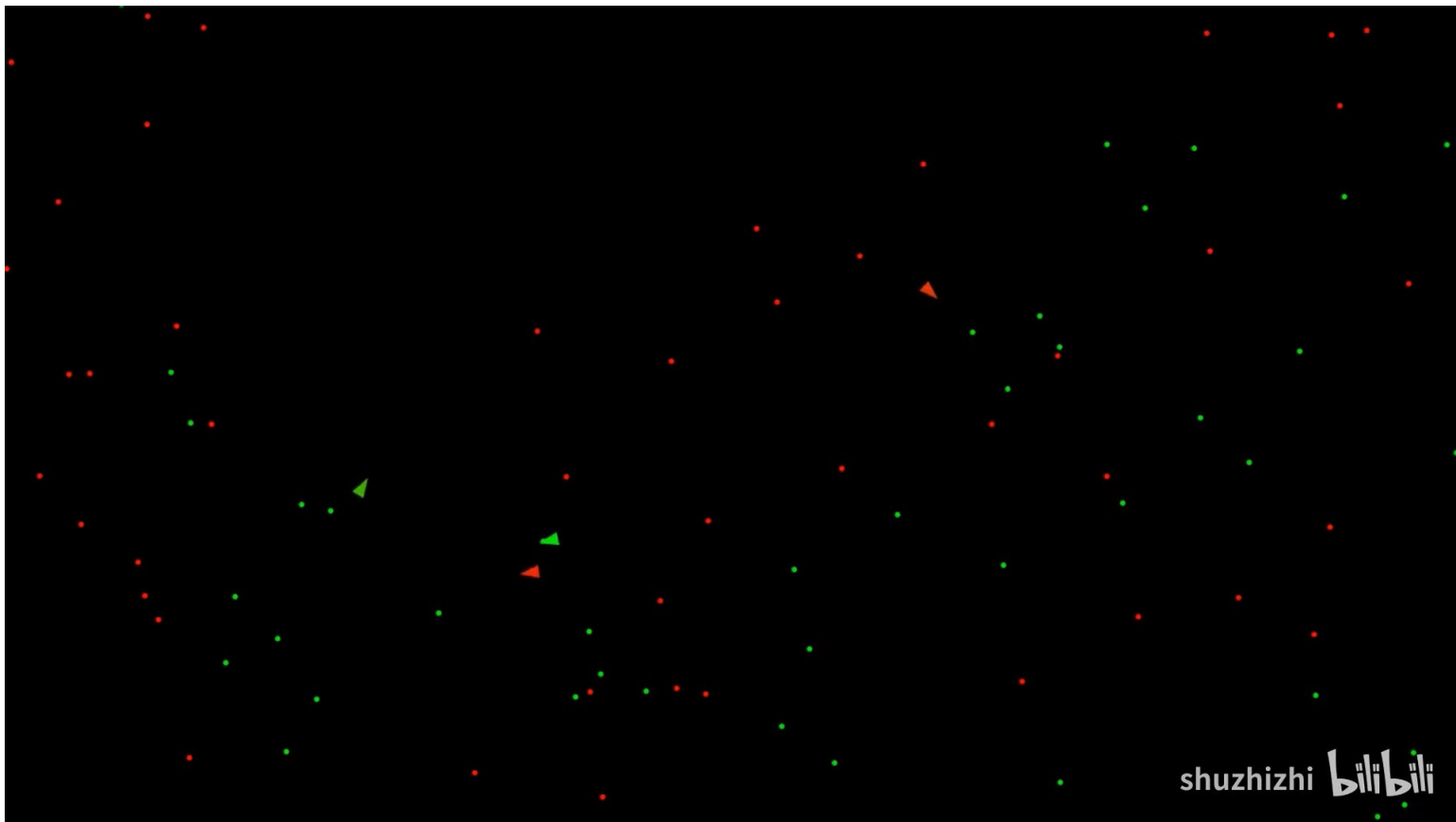
- 6.1 进化算法的产生与发展
- 6.2 遗传算法及其应用
- 6.3 粒子群优化算法及其应用
- 6.4 蚁群算法及其应用(自学)

- 进化算法是一个“算法簇”或“算法框架”，包括遗传算法(GA)、进化策略、进化规划、遗传编程等。
- 进化算法的基本框架是遗传算法所描述的框架。



1. 下面的例子借鉴了生物学的哪些概念？
2. 和上节课讲到的搜索有哪些关联？





什么是遗传算法？

- » **遗传算法** (Genetic Algorithm, GA)：一种模拟生物进化过程的优化算法，由美国密歇根大学的John Holland提出，并在1975年首次详细描述在他的著作《Adaptation in Natural and Artificial Systems》中。
- » 核心思想：通过模拟**自然选择**、**基因交叉和变异**等机制，在解空间中搜索最优解。

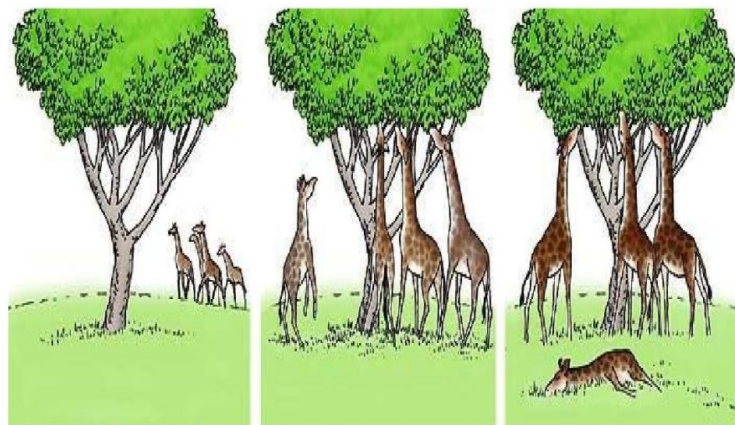
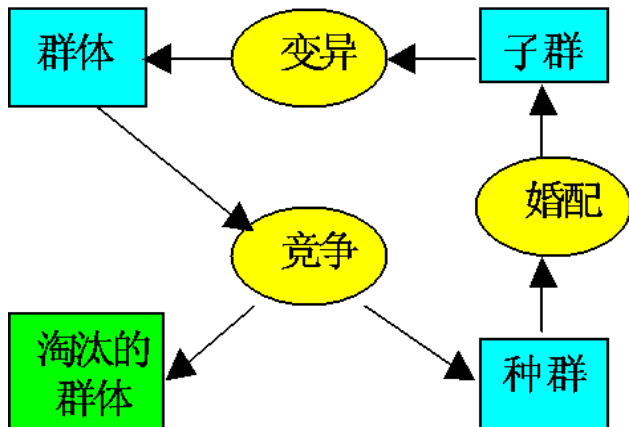


(John Holland, 1929.2.2 -)



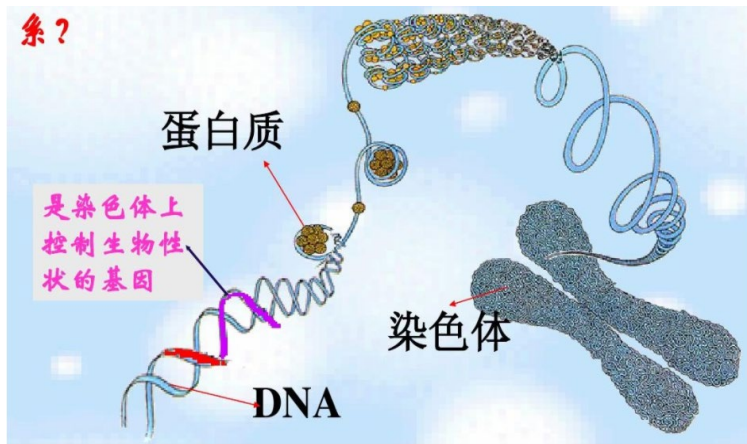
• **遗传算法** (Genetic Algorithms, GA): 一类借鉴生物界**自然选择**和**基因遗传学原理**的**启发式随机搜索算法**。

- **自然选择**: 生物在自然界的生存环境中**适者生存, 不适者被淘汰**的过程。
- **遗传**: 子代总是与亲代 (父代) 相似。(龙生龙, 凤生凤, 老鼠的儿子会打洞)



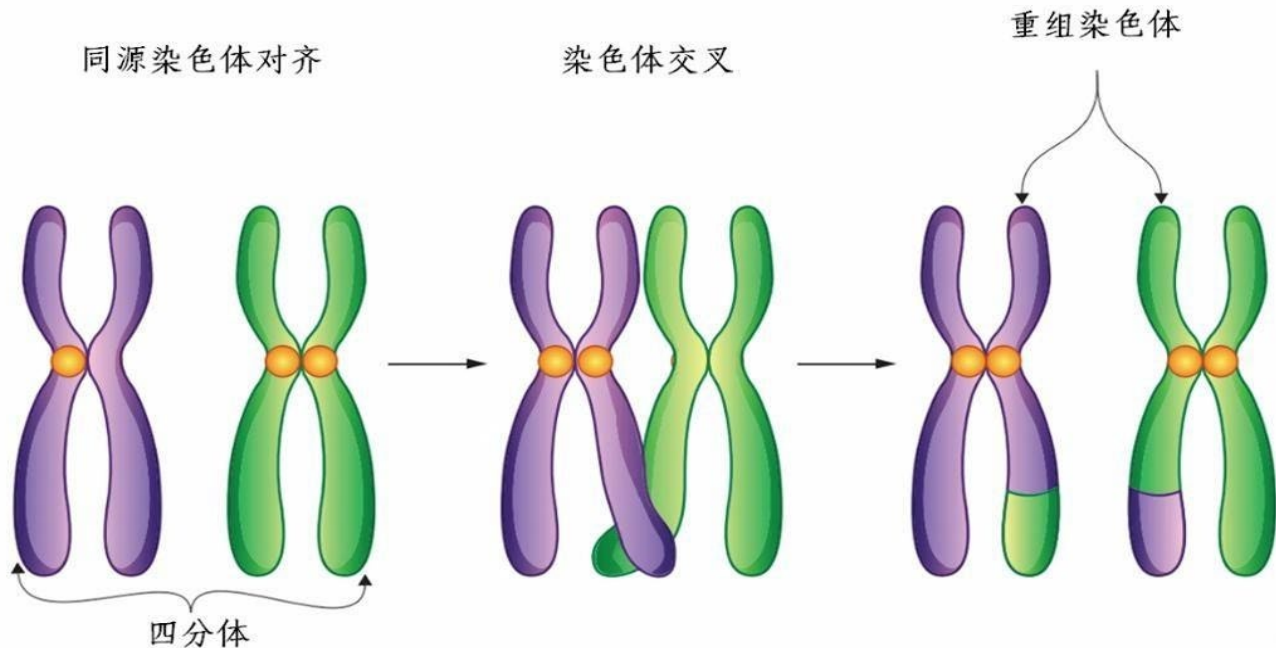
• **遗传算法** (Genetic Algorithms, GA): 一类借鉴生物界**自然选择**和**基因遗传学原理**的启发式随机搜索算法。

- **自然选择**: 生物在自然界的生存环境中**适者生存**, **不适者被淘汰**的过程。
- **遗传**: 子代总是与亲代 (父代) 相似。 (龙生龙, 凤生凤, 老鼠的儿子会打洞)



- **染色体(chromosome)**: 生物的遗传物质的主要载体;
- **基因(gene)**: 扩展生物性状的遗传物质的功能单元和结构单位;
- **基因座(locus)**: 染色体中基因的位置;
- **等位基因(alleles)**: 基因所取的值。

在遗传算法中，主要模拟细胞水平上的基因、染色体和交叉过程等。

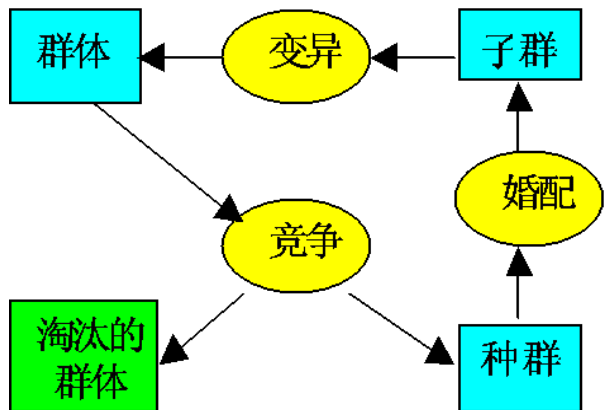


减数分裂过程（将一个生物体的一半遗传物质与另一个生物体的一半遗传物质相结合的过程）

- **遗传算法** (Genetic Algorithms, GA): 一类借鉴生物界**自然选择**和**基因遗传学原理**的**启发式随机搜索算法**。

适应度函数

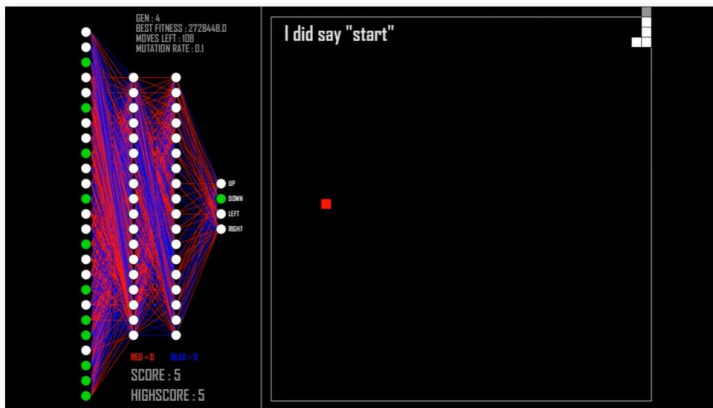
- **实现**: 在计算机上模拟生物的进化过程和基因的操作 (繁殖、重组、变异)
- **特点**: 适用于处理传统搜索方法难以解决的**复杂和非线性优化问题**
- **应用**: 物流路径规划、工程设计优化、机器学习超参数调优、投资组合优化等



- Matlab的GA工具箱 **gatool**
- 谢菲尔德大学的Matlab遗传算法库 **Gatbx** 以及其升级版 **GEATbx**
- python的GA库: **DEAP**、**mealpy**、**scikit-opt**、**Geatpy2**、**GAFT**等
- <https://zhuanlan.zhihu.com/p/487354780>



基于神经网络和遗传算法的Flappy Bird



学赛车

<https://www.bilibili.com/video/av32652759/?p=2>

工程源码地址:

<https://gitee.com/lipengfeikk041kk/AICar>

用遗传算法训练神经网络来玩贪吃蛇

Github代码地址:<https://github.com/greerviau/SnakeAI>

- 遗传算法的生物遗传学基础

- 1) 选择——方向

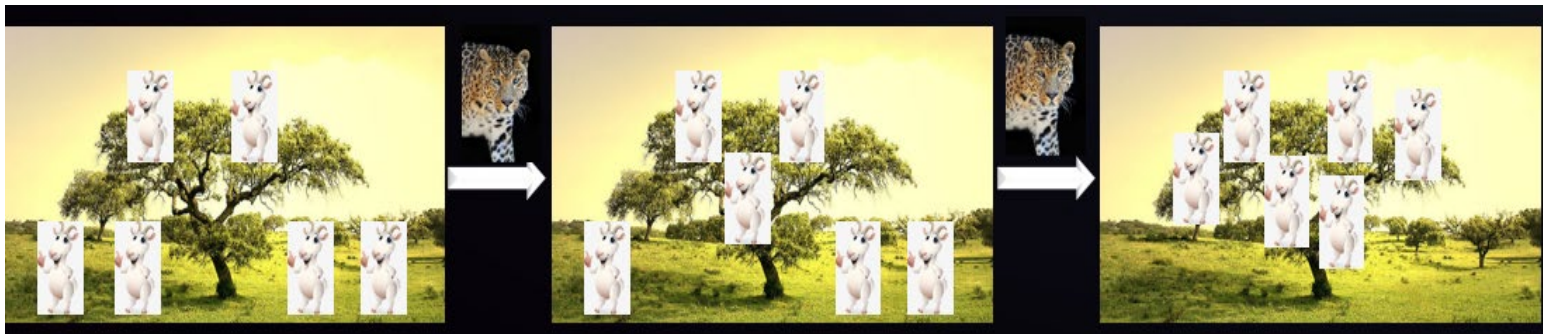
- 2) 遗传——基础

- 3) 变异——条件

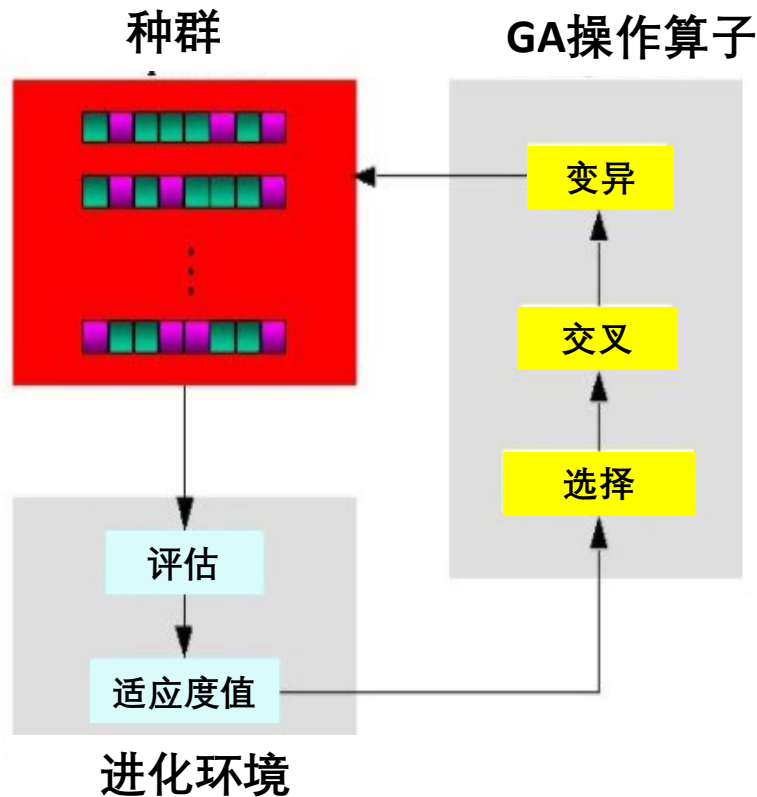
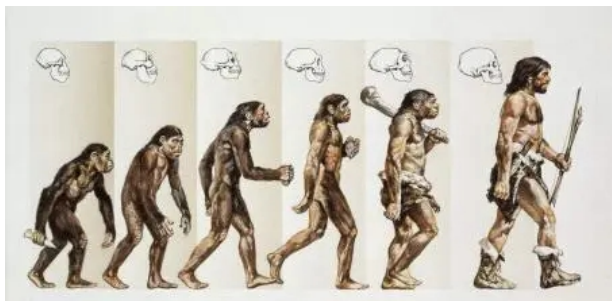
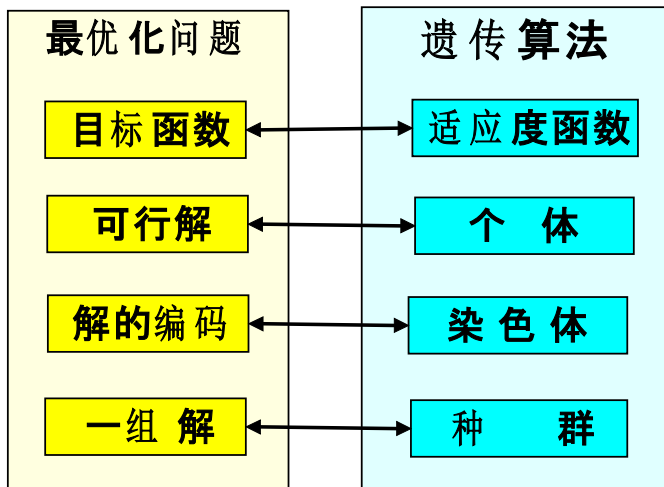
- 选择通过遗传和变异起作用，变异为选择提供资料，遗传巩固与积累选择的资料；

- 选择控制变异与遗传的方向，使变异和遗传向着适应环境方向发展。

问题：自然系统的进化
机制与GA的启发随机搜
索机制对应？



- 遗传算法的基本流程





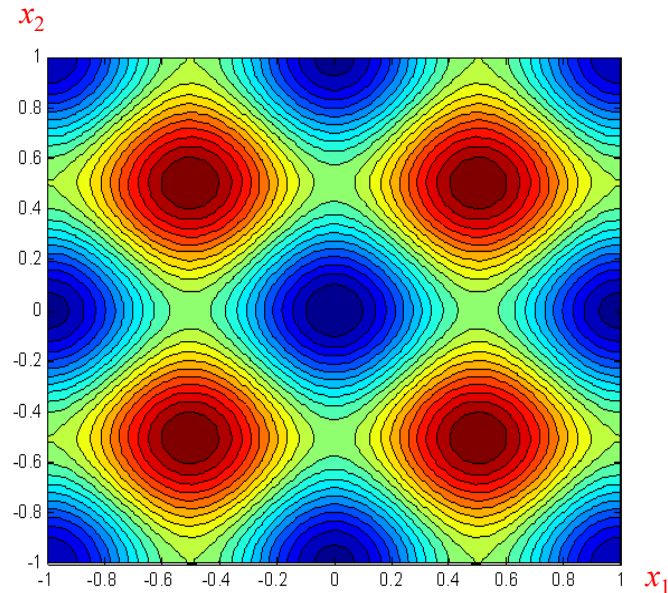
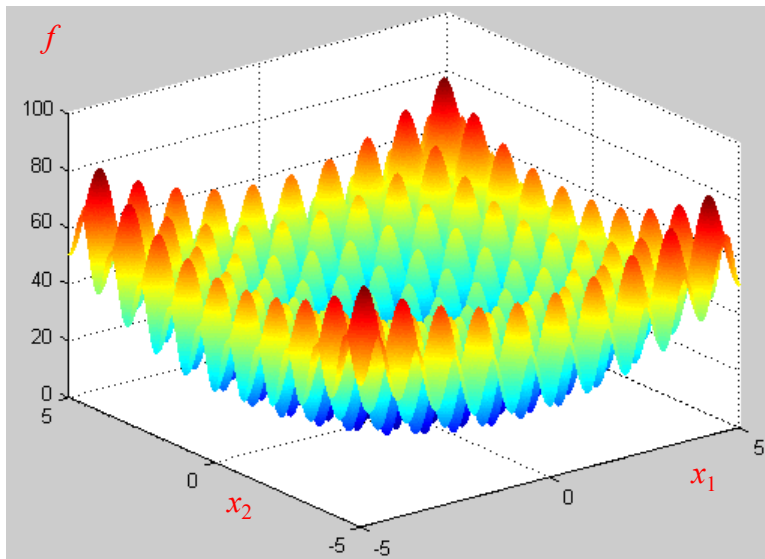
GA的基本要素

- ◆ 解的编码
- ◆ 初始种群的设定
- ◆ 适应度函数的设计
- ◆ 遗传操作（选择、交叉、变异）
- ◆ 算法控制参数的设定
- ◆ 约束条件的处理

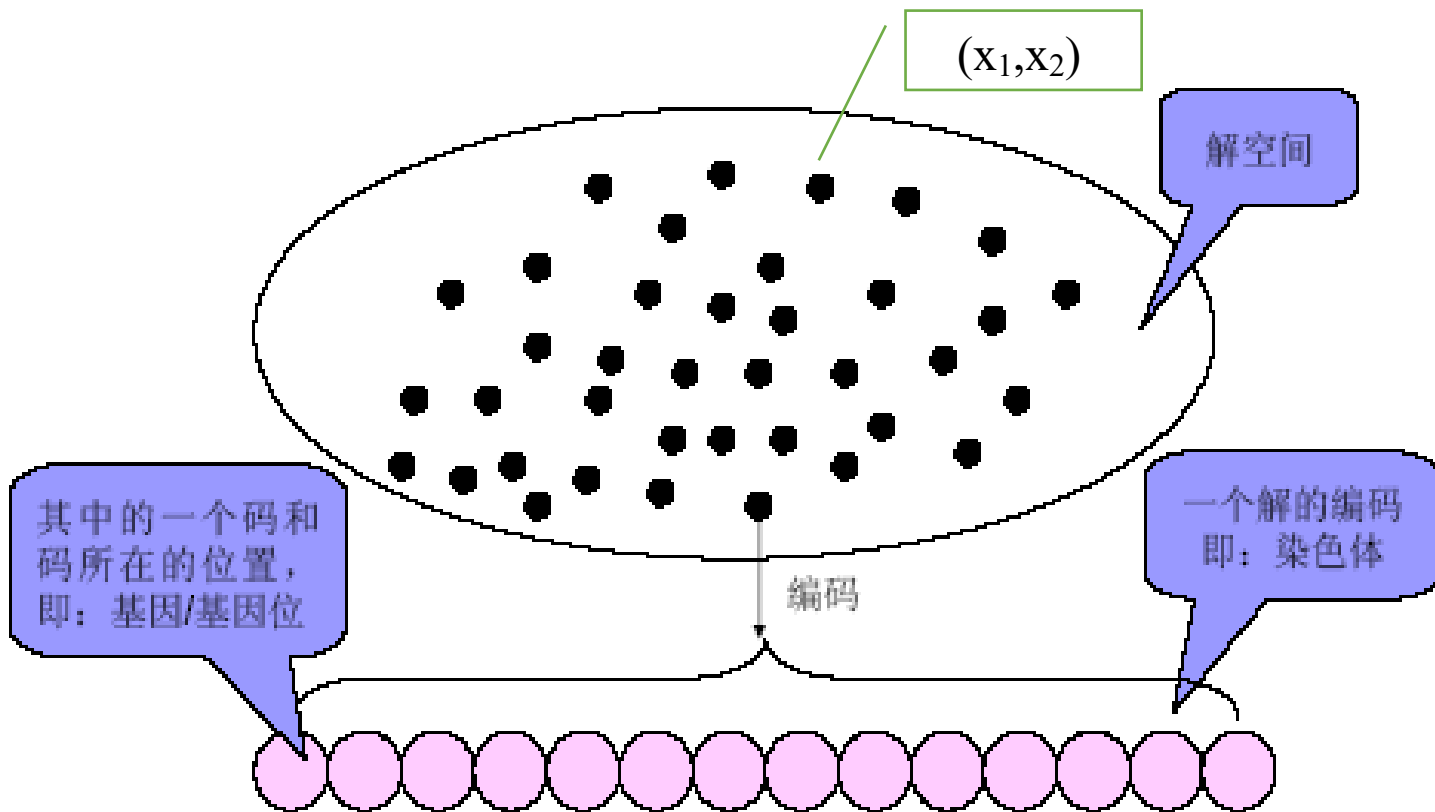
- 重点：
 - GA基本操作：选择、交叉、变异
 - 遗传算法的基本流程
- 难点：
 - 不同的编码方式特点，如何确定适应度函数，不同的交叉操作、变异操作、选择操作之间的区别

□ 例：用遗传算法求解下面一个 Rastrigin 函数的最小值。

$$f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10(\cos 2\pi x_1 + \cos 2\pi x_2)$$
$$-5 \leq x_i \leq 5 \quad i = 1, 2$$



► 解的编码 —— GA的基础

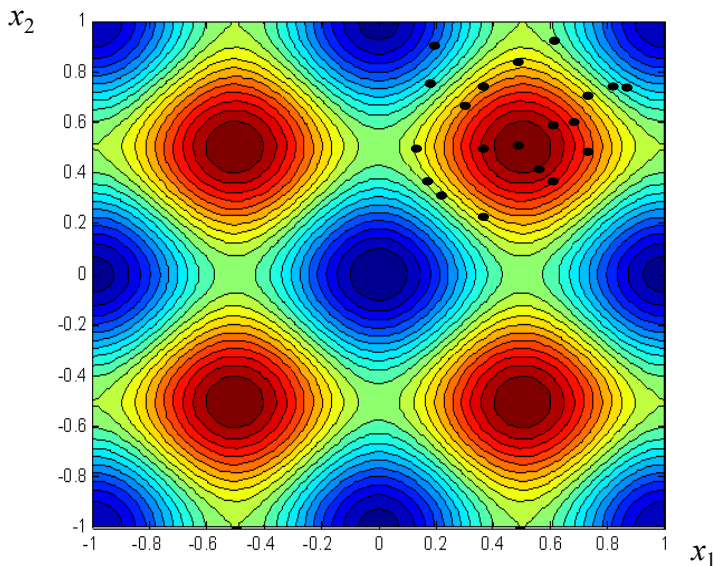


□ 例：用遗传算法求解下面一个Rastrigin函数的最小值。

$$f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10(\cos 2\pi x_1 + \cos 2\pi x_2)$$

$$-5 \leq x_i \leq 5 \quad i = 1, 2$$

► 初始化种群：随机生成20个个体



解的编号

解的编码

1



2



3



4



...

...

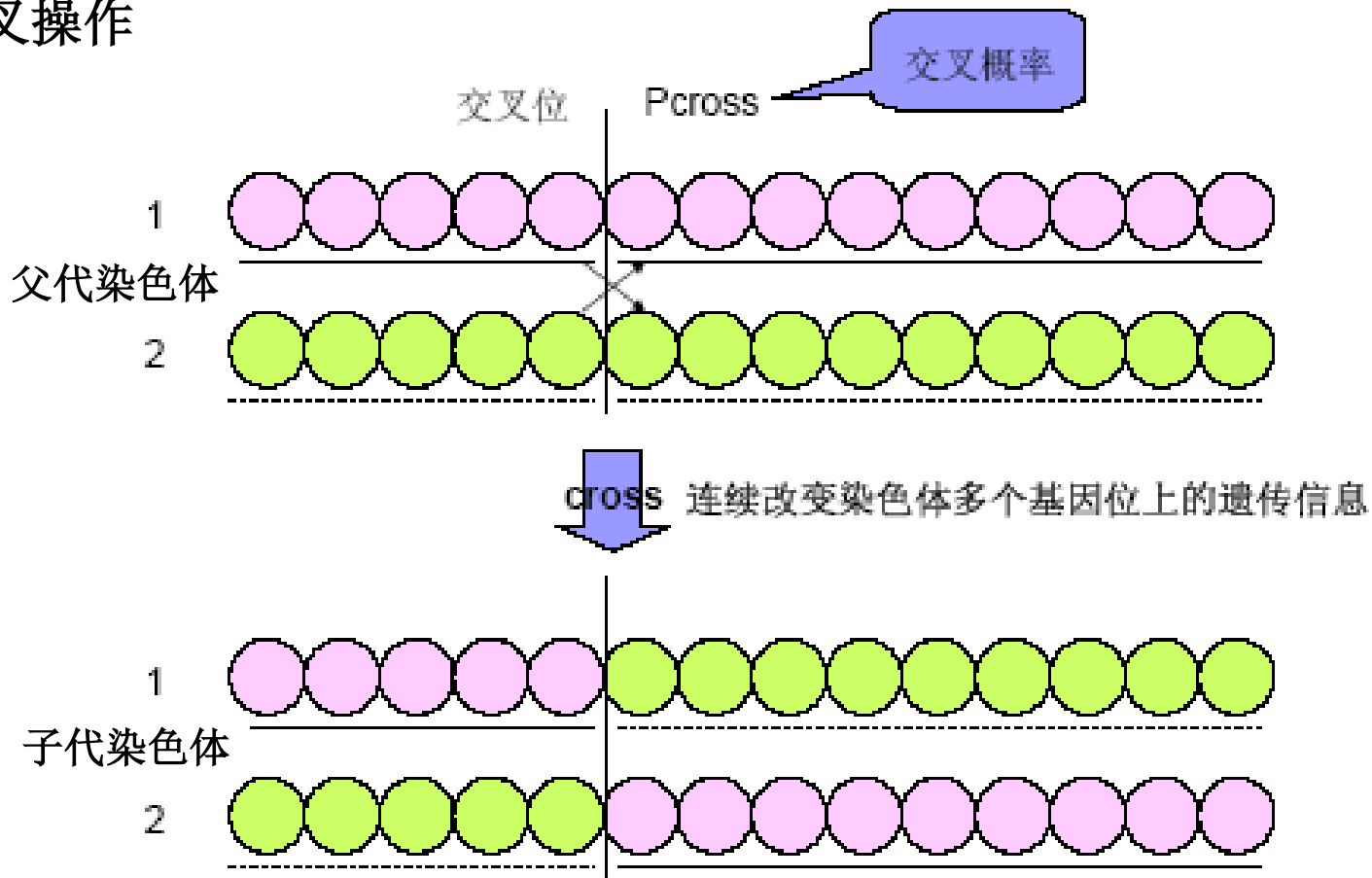
...

POP

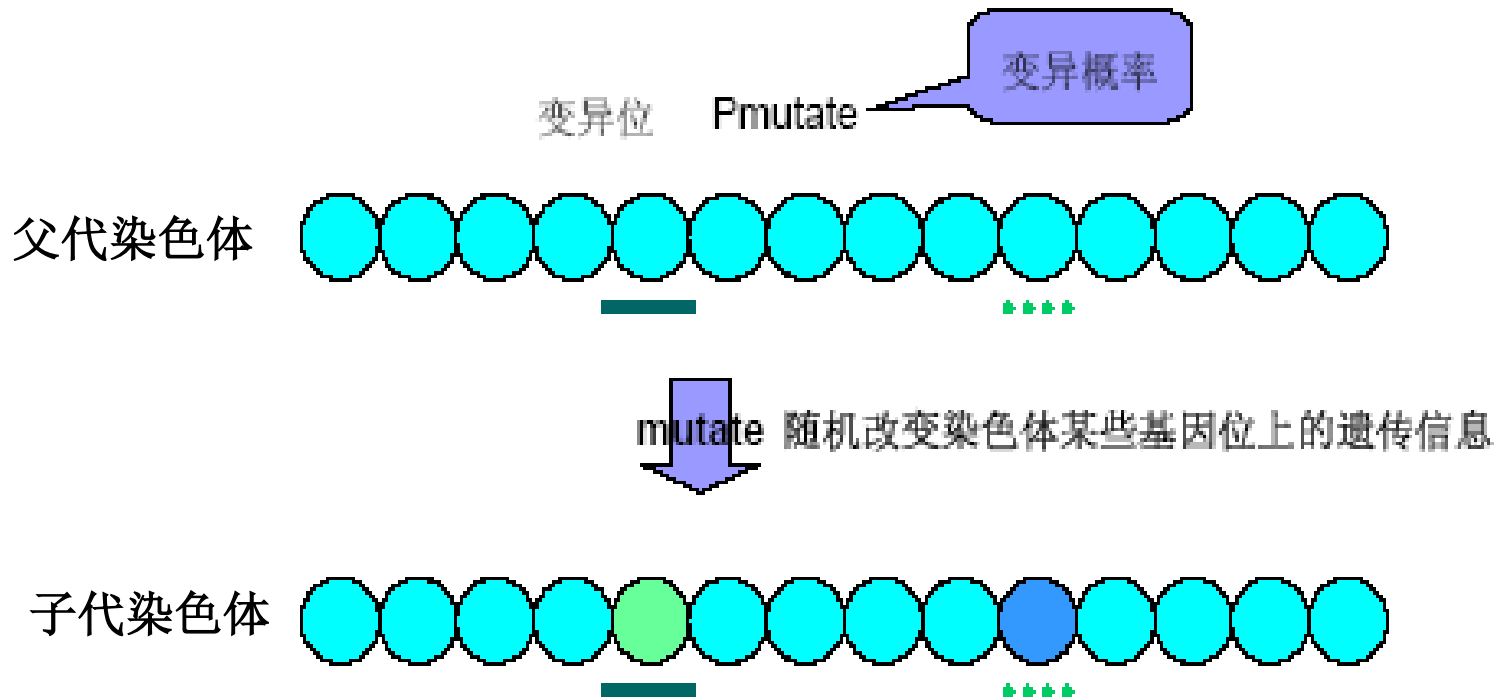
群体规模



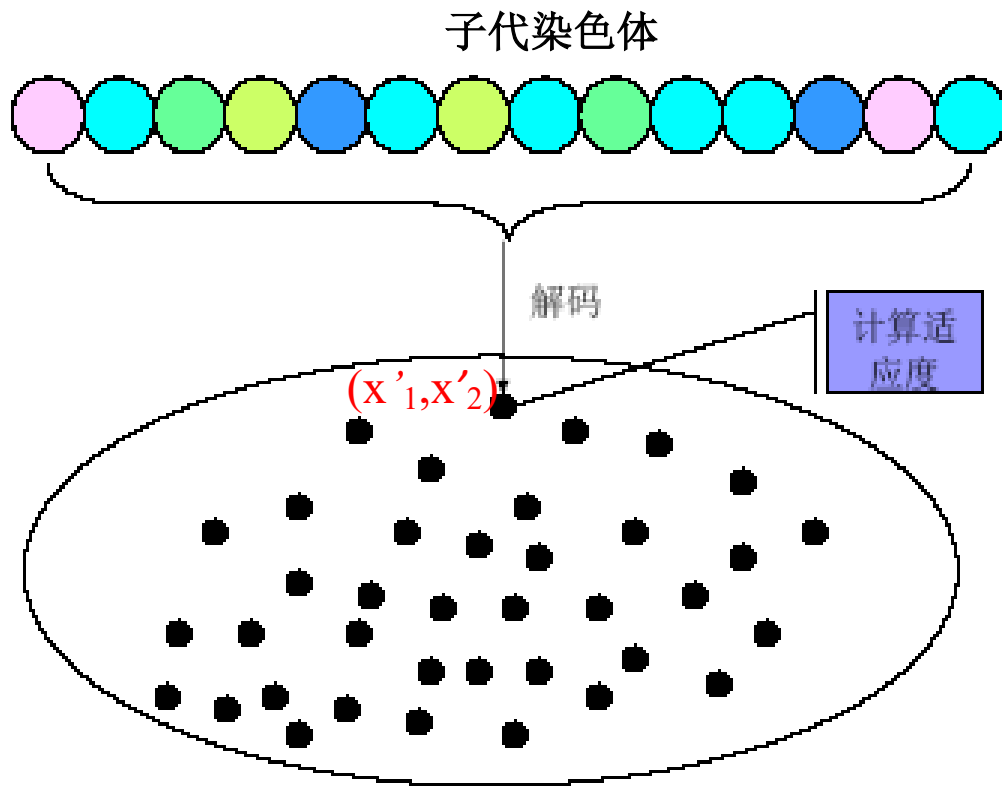
► 交叉操作



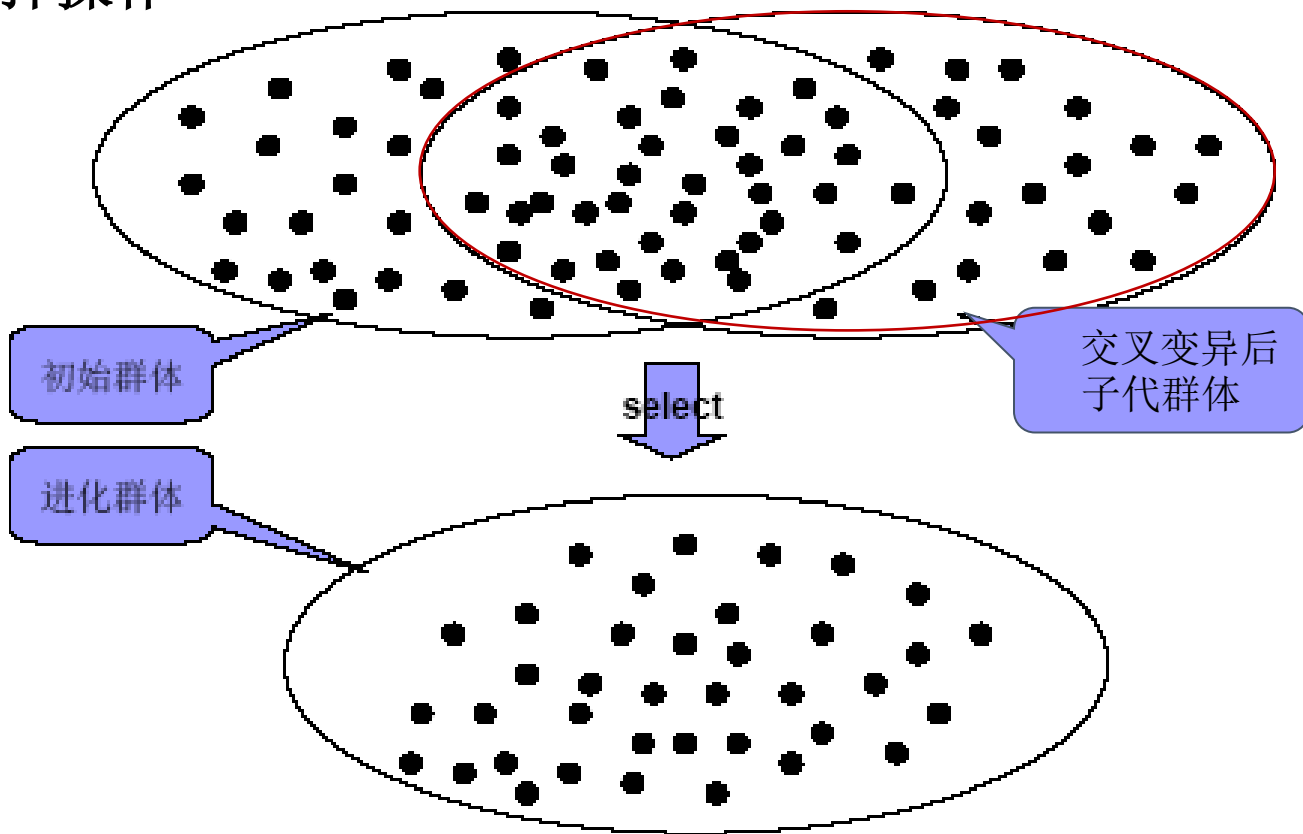
► 变异操作



► 适应度计算



► 选择操作

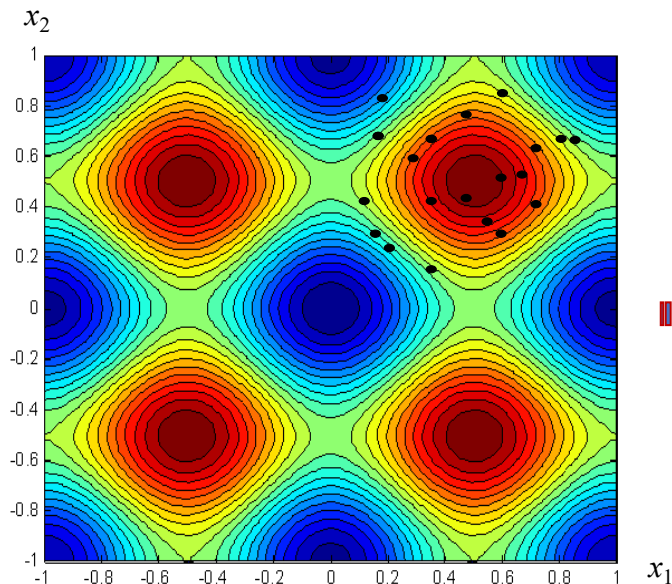


□ 例：用遗传算法求解下面一个Rastrigin函数的最小值。

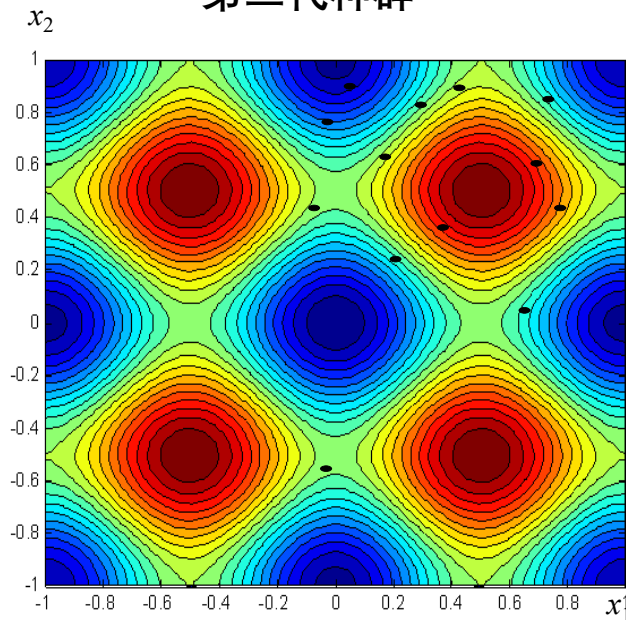
$$f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10(\cos 2\pi x_1 + \cos 2\pi x_2)$$

$$-5 \leq x_i \leq 5 \quad i = 1, 2$$

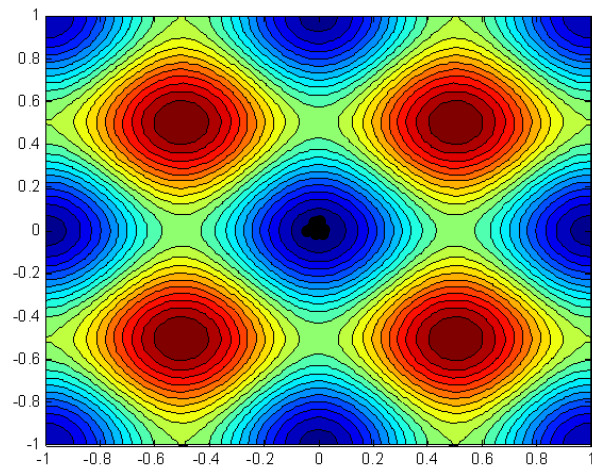
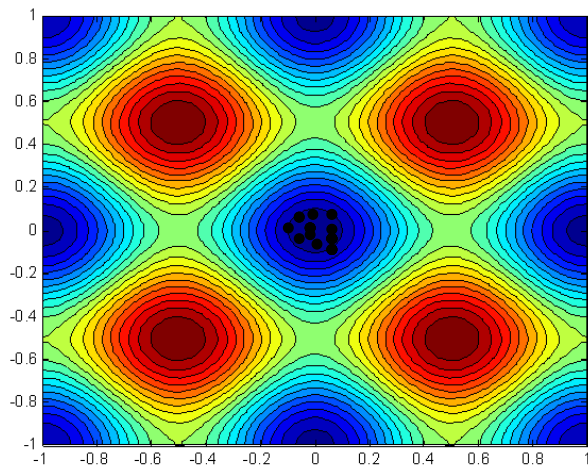
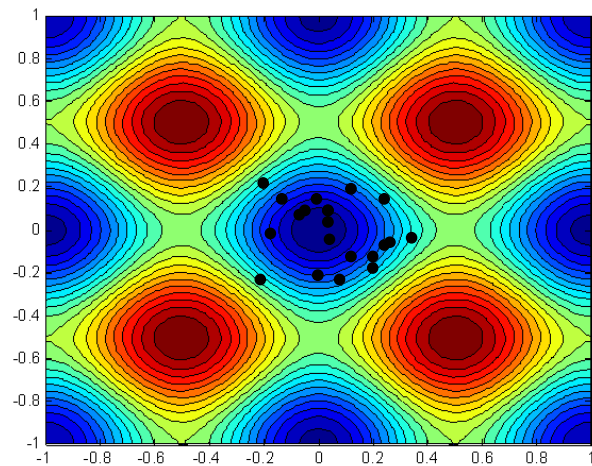
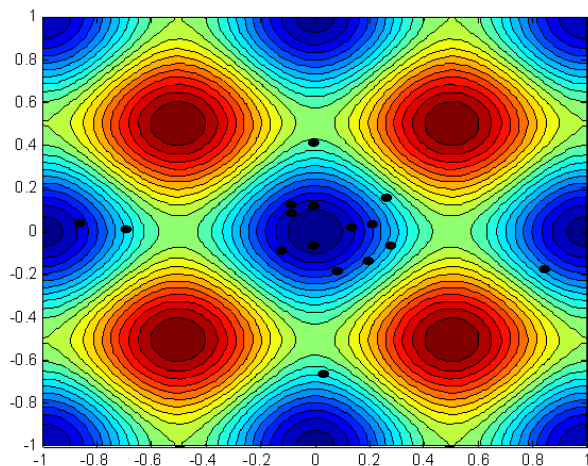
初始种群



第二代种群



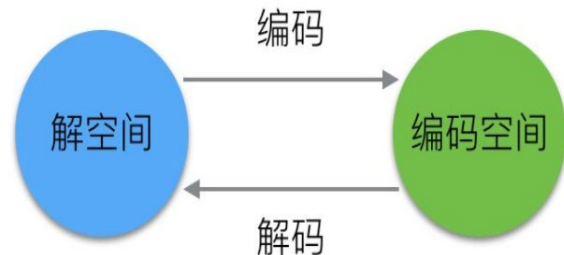
- 在迭代60、80、95、100次时的种群



生物遗传概念	遗产算法中的应用	路径规划问题
个体 (Individual)	解	路径
染色体 (Chromosome)	解的编码（字符串、向量等）	所有路口组成的逻辑 值向量
基因 (Gene)	解的编码中每一分量	每个路口是否经过的 逻辑值
群体 (Population)	根据适应度值选定的一组解（解的个 数为群体的规模）	一组路径方案
适应性 (Fitness)	适应度函数值	路径长短
适者生存	目标值比较大（或小）的解被选择的 可能性大	短路径的选择概率大
婚配 (Marry)	交叉（Crossover）选择两个染色体进 行交叉产生一组新的染色体的过程	向量交叉
变异 (Mutation)	编码的某一分量发生变化的过程	交叉后向量突变

一、编码

- 参数的编码：把一个问题的可行解从其解空间转换到遗传算法所能处理的搜索空间的转换方法。
- 编码对于遗传算子，尤其是对交叉和变异算子的功能和设计有很大的影响。

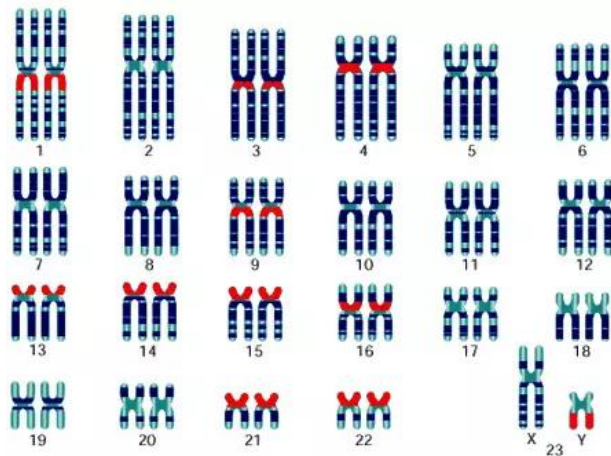
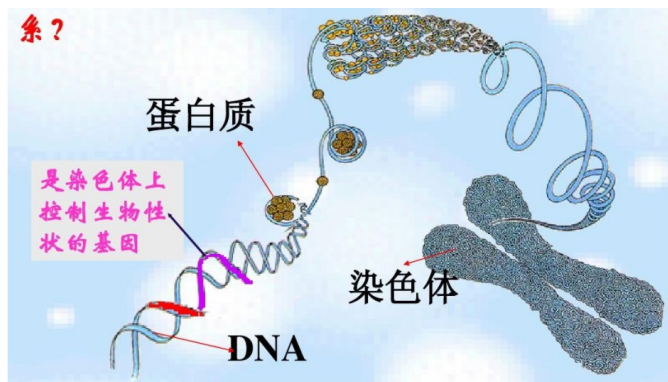


问题的编码一般应满足以下三个规则：

- 完备性(completeness): 原问题空间中的所有点(可行解)都能成为编码后的点；
- 健全性(soundness): 编码后的空间中的点能对应原问题空间所有的点；
- 非冗余性(non-redundancy): 编码前后空间的点一一对应。

一、编码

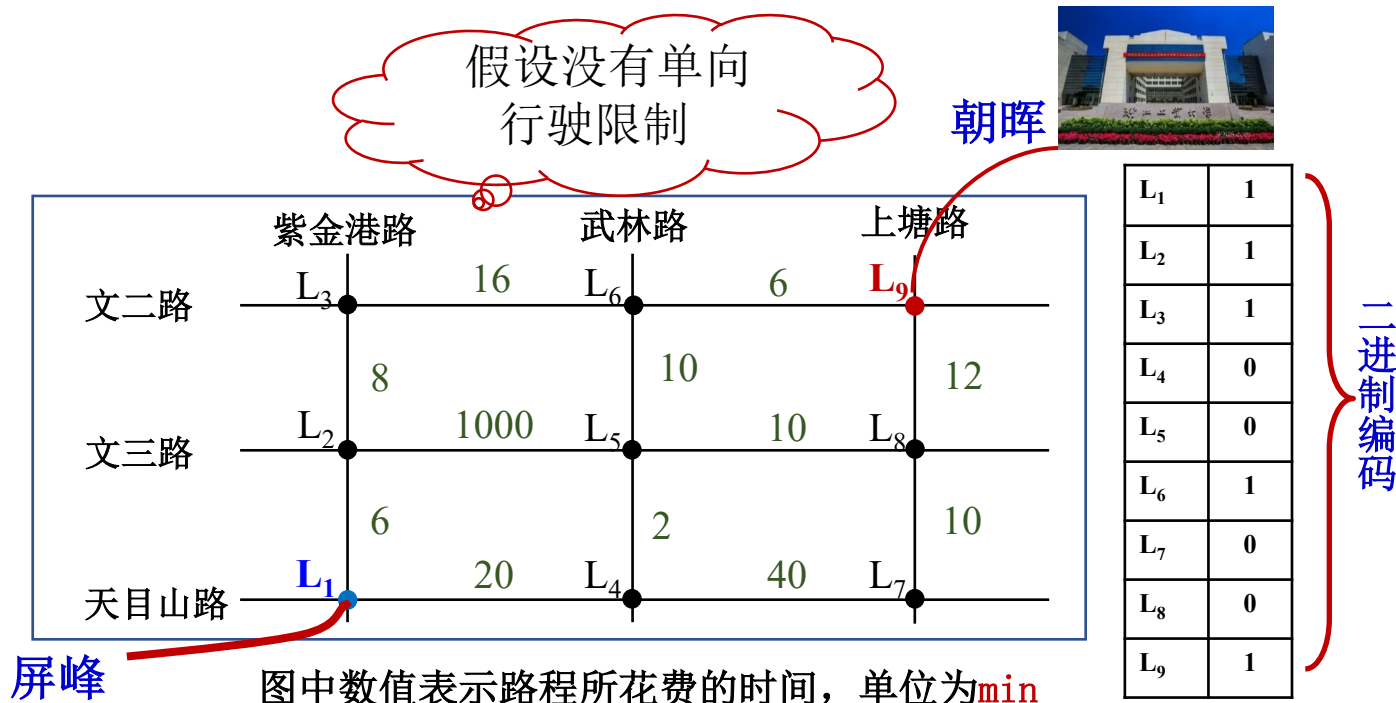
1. **位串编码**（一维染色体编码方法，将问题空间的参数编码为一维排列的染色体的方法。）



□ **二进制编码**：用若干二进制数表示一个个体，将原问题的解空间映射到位串空间 $B = \{0, 1\}$ 上，然后在位串空间上进行遗传操作。

关键：确定染色体（二进制编码）长度！

□ **二进制编码**：用若干二进制数表示一个个体，将原问题的解空间映射到位串空间 $B=\{0, 1\}$ 上，然后在位串空间上进行遗传操作。



路径1: $L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow L_6 \rightarrow L_9$

路径2: $L_1 \rightarrow L_4 \rightarrow L_7 \rightarrow L_8 \rightarrow L_9$

二进制编码呢?



- **二进制编码** (基因用0或1表示, 如: 00100011010)

$$\min f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10(\cos 2\pi x_1 + \cos 2\pi x_2)$$

$$-5 \leq x_i \leq 5 \quad i = 1, 2$$

- **连续问题的二进制串长取决于求解精度**

若 x_1 的求解精度到小数点后1位, 而 x_1 的区间长度为 $5 - (-5) = 10$, 即需将区间分为 $10/0.1 = 100$ 等份。

- $2^6 = 64 < 100 < 2^7 = 128$, 所以 x_1 的二进制编码 A_1 串长为7位。
- 一般 x_i 的编码精度与串长 n_i 关系:

$$\delta_i = \frac{x_{i\max} - x_{i\min}}{2^{n_i}} = \frac{5 - (-5)}{2^7} = 0.078125 \approx 0.1$$

$$A_i \text{ 的解码: } x_i = x_{i\min} + \frac{\text{del}(A_i)}{2^{n_i} - 1} (x_{i\max} - x_{i\min}), \quad \text{del}(A_i) \text{ 把 } A_i \text{ 转为十进制数}$$

0000000: -5

0000001: -4.921260

0000010: -4.842520

0000011: -4.763780

...
1111110: 4.921260

1111111: 5

■ **整条染色体(x_1, x_2)长度呢?**

14位

1. 位串编码（一维染色体编码方法）

■ 二进制编码

- **优点：**类似生物染色体结构，易于用生物遗传理论解释，各种遗传操作易实现。

- **缺点：**

- ①针对连续问题，要先给出求解精度，缺乏微调功能。

- ②高维优化问题的二进制编码串长，算法的搜索效率低。

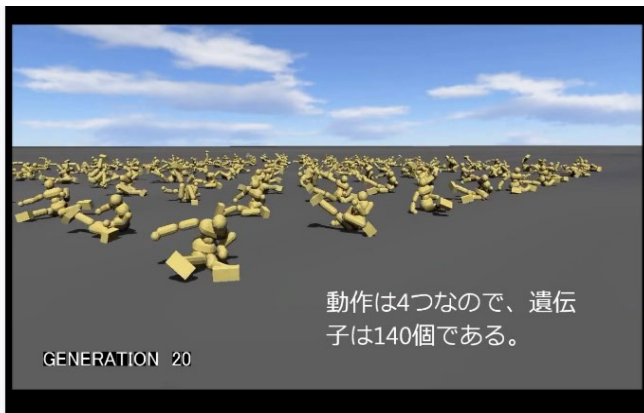
- ③造成Hamming 悬崖，降低遗传算子的搜索效率。

例 7：0111 ， 8： 1000 则 Hamming 距离=4

2. 实数编码

- 优点:

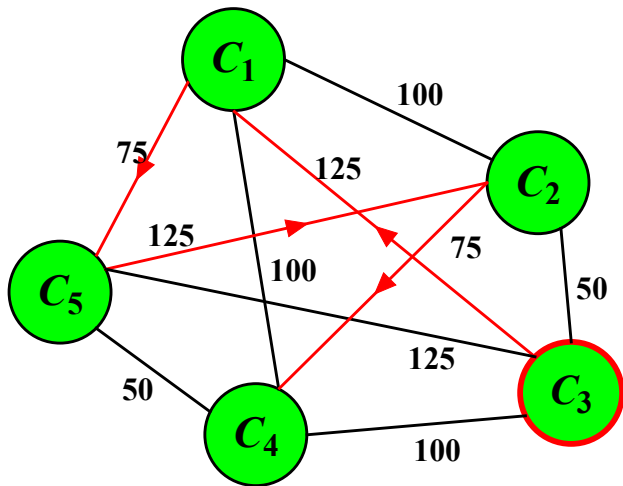
- ① 不必进行数制转换，可直接在解的表现型上进行遗传操作；
- ② 适合于高维或复杂优化问题；
- ③ 适合于精度要求较高的遗传算法。



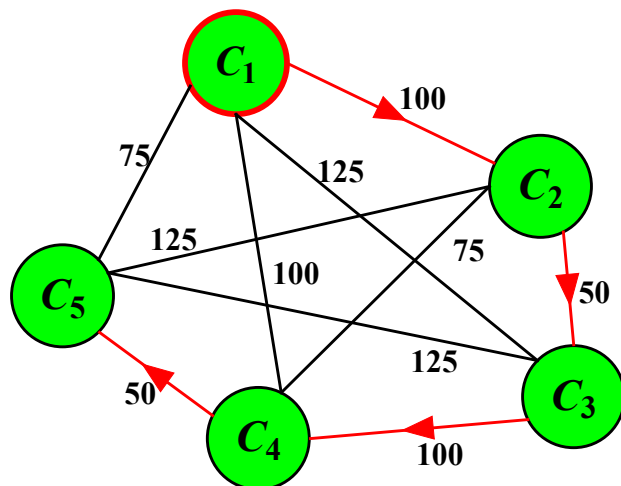
3. 有序串编码

- **有序问题**：目标函数的值不仅与表示解的字符串的值有关，而且与其所在字符串的位置有关。
- 例如 **5个城市的TSP问题的有序串编码**用来表示遍历的城市顺序：

旅行商 (TSP) 问题、作业调度 (job-scheduling)、路由优化、资源调度、物流调度等



3 1 5 2 4: 路径长度为500

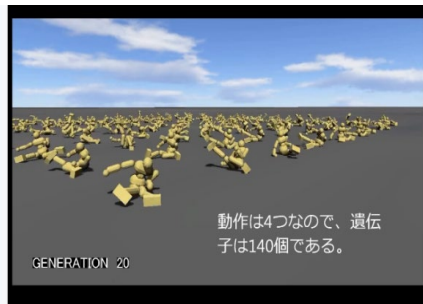


1 2 3 4 5: 路径长度为 375

二、群体设定

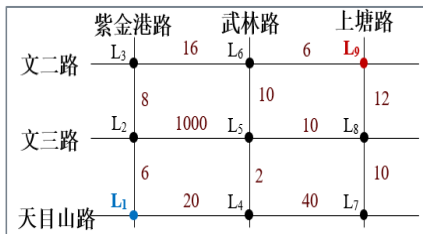
1. 种群规模的确定p145: 需要在多样性和计算成本间权衡

- 规模太小, 易陷入局部最优解, 出现早熟;
- 规模太大, 计算量增加, 影响算法效率。
- 一般种群规模 $N=20\sim 200$ 。问题越难, 种群规模应适当大些。



2. 初始种群的产生p144

- 完全随机的方法: 用随机数发生器产生;
- 基于先验知识(例如最优解在整个问题空间的分布范围)的随机产生方法;
- 先随机产生一些个体, 从中选取 N 个最好的个体作为初始种群。



□ 三、适应度函数（求最大化，非负）设计p145

- 目标函数 $f(x)$ $\rightarrow$ 适应度函数 $Fit(x)$

目标函数 $f(x)$	适应度函数 $Fit(x)$
$\max f(x), \quad f(x) \geq 0$	$Fit(x) = f(x)$ 例：最大化投资收益
$\min f(x), \quad f(x) > 0$	$Fit(x) = \frac{1}{f(x)}$ 例：最小化行驶路径
$\max f(x)$	$Fit(x) = \begin{cases} f(x) - C_{\min} & f(x) > C_{\min} \\ 0 & \text{其他情况} \end{cases}$
$\min f(x)$	$Fit(x) = \begin{cases} C_{\max} - f(x) & f(x) < C_{\max} \\ 0 & \text{其他情况} \end{cases}$

$\min f(x), \quad f(x) \geq 0$ ，适应度函数 $Fit(x)$ 怎么确定？

□ 三、适应度函数（求最大化，非负）设计

✓ 适应度函数设计不当有可能会出**现欺骗问题**(p146, 遗传算法中, 所有妨碍适应度高的个体产生而影响算法正常工作的问题):

过早收敛: 进化**初期**, **个别超级个体**控制选择过程;

停滞现象: 进化**末期**, **个体差异太小**导致陷入局部极值。

✓ 适应度函数的**尺度变换**或者**定标**: 对适应度函数值域的某种映射变换。P138

线性变换:

$$f' = af + b = \frac{(f - f_{\min})f_{\text{avg}}}{f_{\text{avg}} - f_{\min}}$$

原适应度

定标
后的
适应
度

$$a = \frac{f_{\text{avg}}}{f_{\text{avg}} - f_{\min}}$$

$$b = \frac{-f_{\min}f_{\text{avg}}}{f_{\text{avg}} - f_{\min}}$$

满足 $f'_{\text{avg}} = f_{\text{avg}}$, f' 非负

缩小超级个体的适应度, 以降低这些超级个体的竞争力

改变原始适应值的比例关系, 以提高个体之间的竞争力

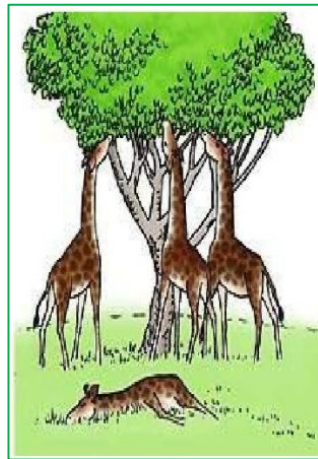
四、选择（复制）

- 1. 个体选择概率分配方法
- (1) 适应度比例方法(fitness proportional model) 或蒙特卡罗法(Monte Carlo)

- 个体的选择概率 p_i 和其适应度值 f_i 成比例:

$$p_i = \frac{f_i}{\sum_{i=1}^N f_i}$$

如果适应度值没有
定标，易出现过早收敛
和停滞现象



选择操作的主要目的是在保持种群大小恒定的情况下复制种群中适应度高的个体，去除种群中适应度低的个体。

选择操作：确定从亲代种群中按某种方法选取哪些个体遗传到下一代种群中

- 1. 个体选择概率分配方法
- (2) 排序方法 (rank-based model) **p148**

克服适应度值比例
选择策略的过早收
敛和停滞现象

① 线性排序: J. E. Baker

- 个体按适应度值从大到小依次排列: $x_1, x_2, \dots, x_N$
- 个体 x_i 分配选择概率 $p_i = \frac{a - bi}{N(N+1)}$ (a, b 是常数, N 为种群规模)

② 非线性排序: Z. Michalewicz

- 将群体成员按适应值从大到小依次排列, 并按下式分配选择概率:

$$p_i = \begin{cases} q(1-q)^{i-1} & i = 1, 2, \dots, N-1 \\ (1-q)^{N-1} & i = N \end{cases}$$

q 为最好个体
的选择概率

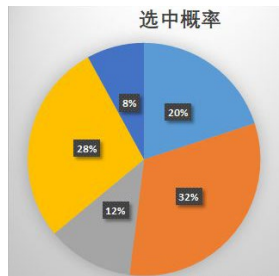
- 1. 个体选择概率分配方法
- 排序方法 v.s 适应度比例法

对比要素	适应度比例方法	排序方法
选择概率	根据适应度计算出来的	事先设计好的，取决于排序
适应度缺陷	容易出现过早收敛和停滞现象	克服了过早收敛和停滞现象
适应度计算	需要进行适应度值的尺度变换 或者定标	不需要进行适应度值的尺度 变换或者定标
健壮性		具有更好的健壮性（鲁棒性）

四、选择（复制）

2. 选择个体方法

1) 转盘赌选择（roulette wheel selection）



个体	1	2	3	4	5	6	7	8	9	10	11
适应度	2.0	1.8	1.6	1.4	1.2	1.0	0.8	0.6	0.4	0.2	0.1
选择概率	0.18	0.16	0.15	0.13	0.11	0.09	0.07	0.06	0.03	0.02	0.0
累积概率	0.18	0.34	0.49	0.62	0.73	0.82	0.89	0.94	0.97	0.99	1.00

$$p_i = \frac{f_i}{\sum_{i=1}^N f_i}$$

• 第1轮产生一个随机数：0.81

• 第2轮产生一个随机数：0.32

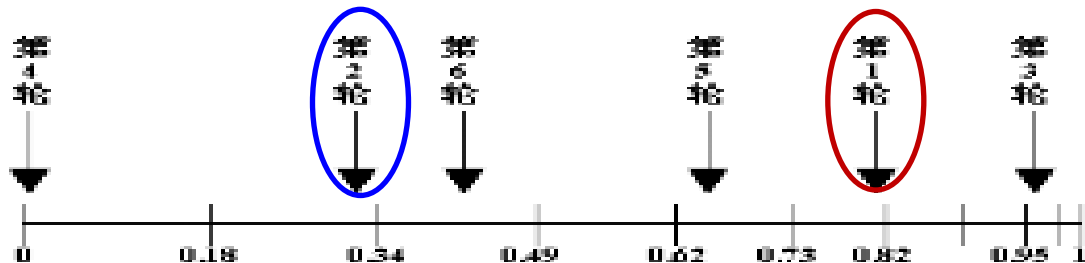
$$Q_1 = p_1 = 0.18$$

$$Q_2 = Q_1 + p_2 = 0.18 + 0.16 = 0.34$$

$$Q_3 = Q_2 + p_3 = 0.34 + 0.15 = 0.49$$

...

$$Q_{11} = Q_{10} + p_{11} = 0.99 + 0.0 \approx 1$$



2. 选择个体方法

2) 锦标赛选择(tournament selection model)

- 从群体中随机选择 k 个个体，将其中**适应度最高的个体**保存到下一代。这一过程反复执行，直到保存到下一代的个体数达到预先设定的数量为止。



- 优点：
- 能得到更加多样化的群体
- 能避免超级个体的影响，一定程度避免了过早收敛和停滞现象的发生

2. 选择个体方法

3) (μ, λ) 和 $\mu + \lambda$ 选择

- (μ, λ) 选择: 从规模为 μ 的群体所生成的 $\lambda (\geq \mu)$ 个后代中选取 μ 个最优的后代作为新一代种群。
- $\mu + \lambda$ 选择: 从 λ 个后代与其父代中选取 μ 个最优的后代作为新一代种群。

4) 最佳个体 (elitist model) 保存 (精英选择)

- 把群体中适应度最高的一个或多个 (2%~5%) 个体直接复制到下一代中, 保证遗传算法终止时得到的最后结果一定是历代出现过的最高适应度的个体。

□ 五、交叉（基因重组） p156

1. 基本的交叉算子（二进制编码）

1) 一点交叉（单点交叉）

$$\begin{array}{lcl}
 A_1 = 0\ 1\ 1\ 0\ \color{red}{1} & \longrightarrow & A_1 = 0\ 1\ 1\ 0\ \color{red}{0} \\
 A_2 = 1\ 1\ 0\ 0\ \color{red}{0} & & A_2 = 1\ 1\ 0\ 0\ \color{red}{1}
 \end{array}$$

2) 二点交叉（两点交叉）

$$\begin{array}{lcl}
 A_1 = 0\ 1\ \color{red}{1\ 0}\ 1 & \longrightarrow & A_1 = 0\ 1\ \color{red}{0\ 0}\ 0 \\
 A_2 = 1\ 1\ \color{red}{0\ 0}\ 0 & & A_2 = 1\ 1\ \color{red}{1\ 0}\ 1
 \end{array}$$

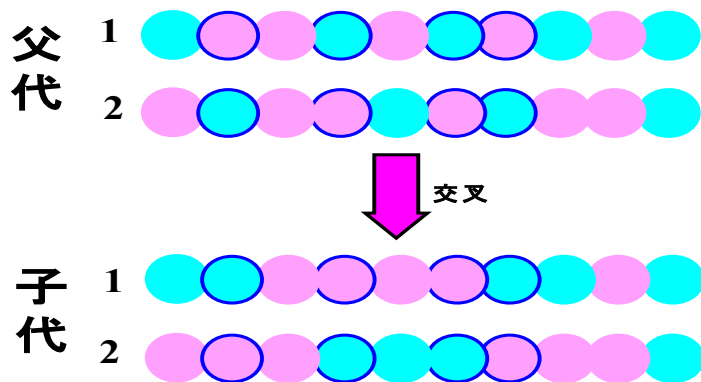
交叉操作：模仿自然界有性繁殖的**基因重组**，是GA产生新个体的主要手段。对两个父代个体进行基因操作，作用是**把原有优良基因遗传到下一代种群中，并生成包含更复杂基因结构的新个体**。

❑ 五、交叉（基因重组） p156

1. 基本的交叉算子（二进制编码）

3) 均匀交叉（uniform crossover)或一致交叉)

两个父代个体的每一个基因位上的基因进行等概率交换而生成两个子代



2. 改进的交叉算子（有序串编码）

1) **部分匹配交叉PMX**: Goldberg D. E. 和R. Lingle(1985)

2) **顺序交叉OX**: Davis L.(1985)

交叉算子的设计和实现与所研究的问题密切相关，一般要求它既**不要太多地破坏个体编码串中表示优良性状的优良模式**，又要能够有效地产生出一些较好的新个体模式。

交叉区的数字移至起点位置，其后预留等于H数目的空间，然后将其余数字按其相对次序排列在预留区后

PMX趋向于所期望的绝对城市位置，OX趋向于所期望的相对城市位置

遗传算法交叉操作演示 9个城市的TSP问题

部分匹配交叉PMX

A = 1 3 4 | 8 5 6 | 7 0 2

B = 0 5 4 | 1 6 7 | 3 8 2

A' = 8 3 4 | 1 6 7 | 5 0 2

B' = 0 7 4 | 8 5 6 | 3 1 2

顺序交叉OX

A = 0 4 1 | 6 5 2 | 8 7 3

B = 3 7 0 | 5 6 8 | 4 1 2

A' = 0 4 1 | H H 2 | H 7 3

B' = 3 7 0 | H H 8 | 4 1 H

A'' = 2 | H H H | 7 3 0 4 1

B'' = 8 | H H H | 4 1 3 7 0

A''' = 2 | 5 6 8 | 7 3 0 4 1

B''' = 8 | 6 5 2 | 4 1 3 7 0

预留区

随机产生 下一步

六、变异

将个体染色体编码串中某些基因座上的基因值用该基因座的其他等位基因来替换，从而形成一个新的个体。

遗传算法变异操作演示 8个城市的TSP问题

两点互换	相邻互换
1 0 2 <u>3</u> 4 5 6 7	0 2 1 <u>4</u> <u>3</u> 5 6 7
1 0 2 <u>7</u> 4 5 6 <u>3</u>	0 2 1 <u>3</u> <u>4</u> 5 6 7
下一步	下一步

区间逆转	单点移动
<u>0</u> 3 2 1 4 6 5 7	0 1 2 3 4 5 6 7
<u>5</u> 6 4 1 2 3 <u>0</u> 7	0 1 2 3 4 6 5 7
下一步	下一步

实数编码的交叉和变异操作：
王万良，《人工智能及其应用》
第四版

□ 七、控制参数选择

- ◆ 控制参数：种群规模 N ，交叉概率 p_c ，变异概率 p_m 等。
- ◆ 参数选择应该视解决的具体问题而定，到目前为止，还没有一个适用于遗传算法所有应用领域的关于算法参数的理论。

- ◆ 常用的参数范围： $N=20\sim 200$;
 $p_c=0.85\sim 1.0$;
 $p_m=0.001\sim 0.15$

p_c 越大，产生新个体的速度就越快；
 p_c 过大，组块被破坏的概率也越大；
 p_c 过小，使搜索变慢、停滞。

p_m 过小，不易产生新个体；
 p_m 过大，变成纯粹的随机搜索算法。

选择的作用：优胜劣汰、适者生存；

交叉的作用：保证种群稳定性，朝着最优解的方向进化；

变异的作用：保证种群多样性，避免交叉可能产生的局部收敛。

1. 若用遗传算法求解下面一个非线性函数的最小值，假定求解的精度为小数点后3位：

$$f(x_1, x_2) = 100(x_1^2 - x_2)^2 + (1 - x_1)^2$$
$$-2.048 \leq x_i \leq 2.047 \quad i = 1, 2$$

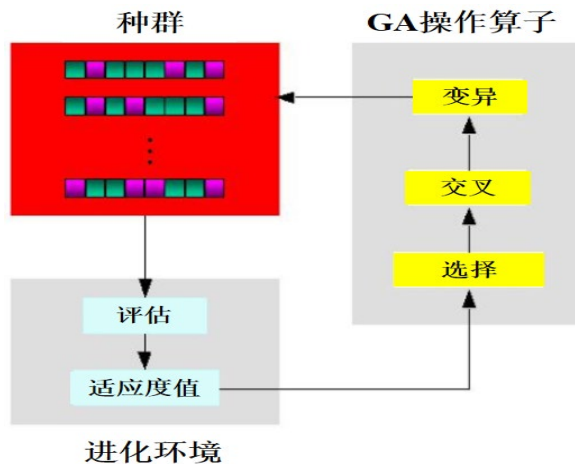
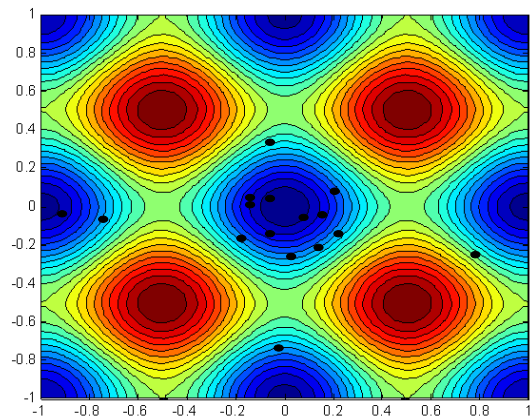
- (1) 给出求解上述问题的遗传算法的二进制编码长度，即一条染色体的长度。
- (2) 给出适应度函数，并说明适应度函数在遗传算法中的作用。

2. 对10个城市的TSP问题采用部分匹配交叉，如果父代1和父代2如下所示，其中|为交叉位，请给出交叉后的子代1和子代2。

父代1: 9035|812|746; 父代2: 0839|571|426

GA的特点:

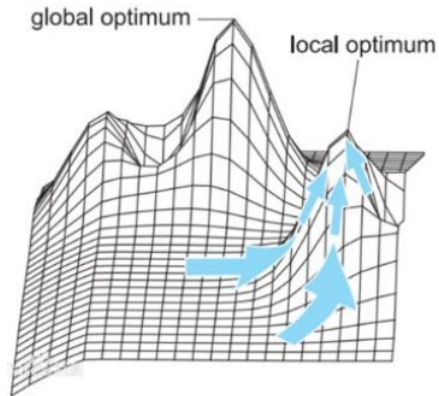
- 只对参数集的**编码**进行操作, 而非对参数本身进行操作;
- 从许多初始点开始**并行搜索**, 搜索的全局性强;
- 通过**目标函数**计算适应度, 对问题的依赖性小;
- 寻优规则是由**概率**决定的, 而非确定性的;
- 在解空间进行高效**启发式**搜索, 非盲目地穷举或完全随机搜索;
- 具有**并行计算**的特点, 适合于复杂问题的优化;
- **计算简单, 功能强大**。



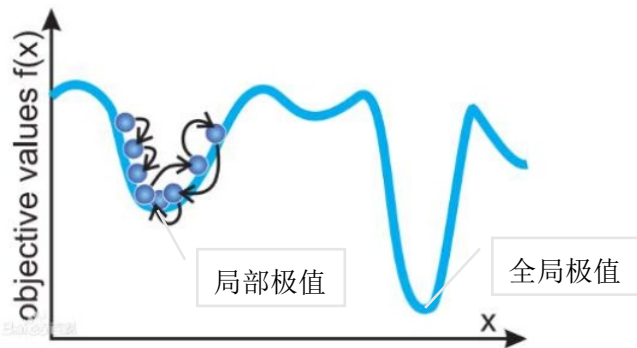
- 双倍体遗传算法
- 双种群遗传算法
- 自适应遗传算法

■ 基本遗传算法（启发式的随机的全局搜索方法）的主要不足：

- 早熟/停滞：局部收敛
- 稳定性差、参数敏感



早熟/停滞：局部收敛



1. 基本思想

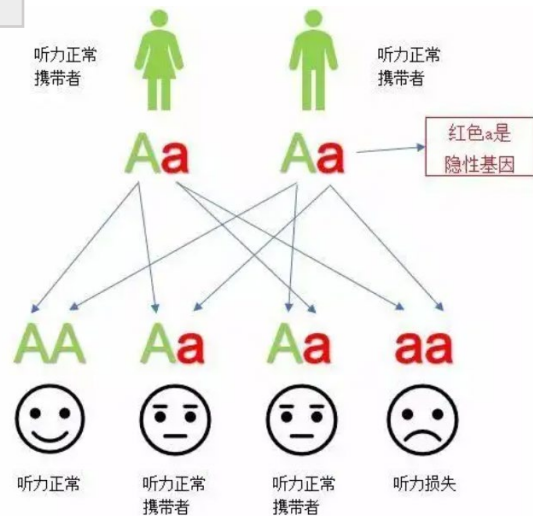
- 双倍体遗传算法采用显性和隐性两个染色体同时进行进化，提供了一种记忆以前有用的基因块的功能。
- 双倍体遗传算法采用显性遗传。

显性染色体：适应值较大的染色体
隐性染色体：适应值较小的染色体

按照显性染色体的选择概率进行选择操作



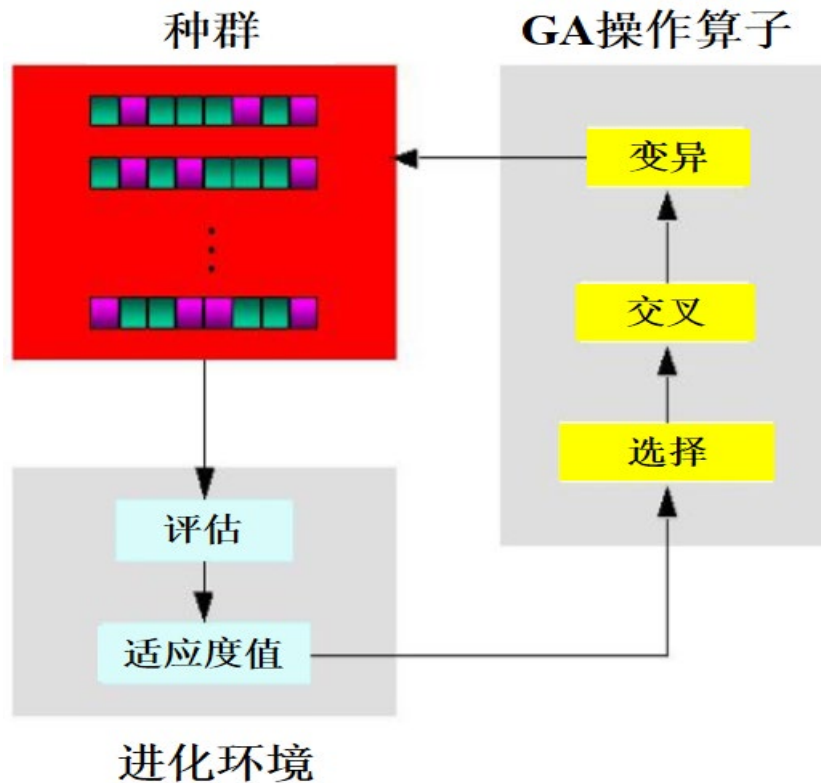
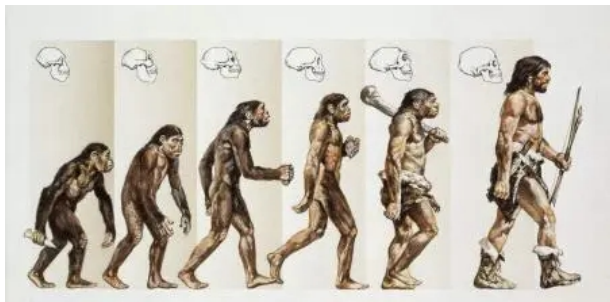
- 双倍体遗传延长了有用基因块的寿命，提高了算法的收敛能力，在变异概率低的情况下能保持一定水平的多样性。



2. 算法流程

- (1) 编码：两个染色体（显性、隐性）
- (2) 复制(选择)算子：计算显性染色体的适应度，按照显性染色体的选择概率将个体复制到下一代群体中。
- (3) 交叉算子：两个个体的显性染色体交叉、隐性染色体也同时交叉。
- (4) 变异算子：个体的显性染色体按正常的变异概率变异；隐性染色体按较大的变异概率变异。
- (5) 显隐性重排算子：个体中适应值较大的染色体设为显性染色体，适应值较小的染色体设为隐性染色体。
- (6) 重复（2） - （5），直至满足终止条件。

- 双倍体遗传算法
- 双种群遗传算法
- 自适应遗传算法



1. 算法思想：建立两个遗传算法群体，分别独立地运行复制、交叉、变异操作，同时当**每一代运行结束以后**，选择两个种群中的**随机个体**及**最优个体**分别交换。

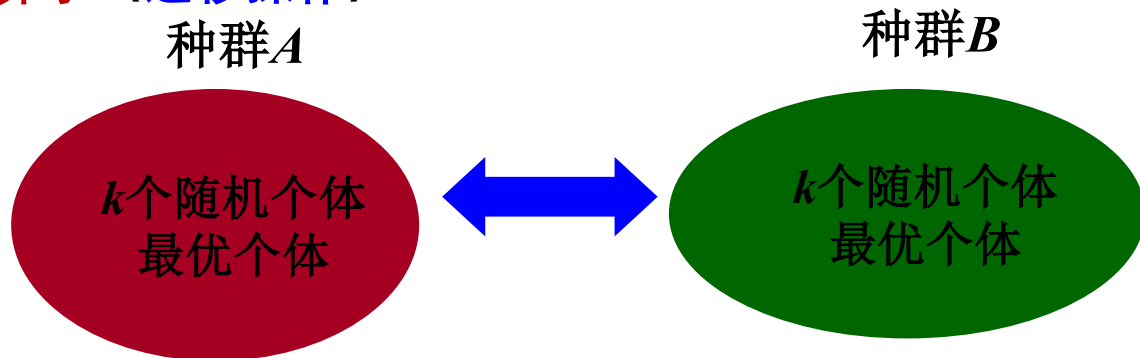
2. 算法设计

(1) 编码设计

(2) 交叉算子、变异算子

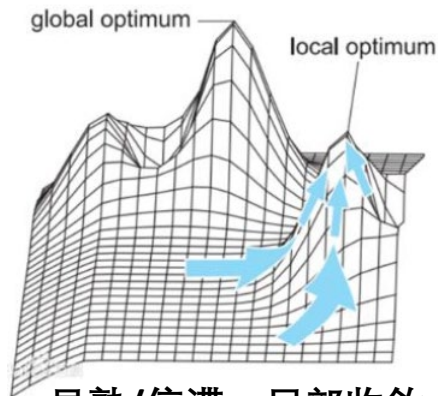
(3) **杂交算子**（**迁移操作**）

打破原种群内的平衡态达到更高的平衡态，有利于算法跳出局部最优。



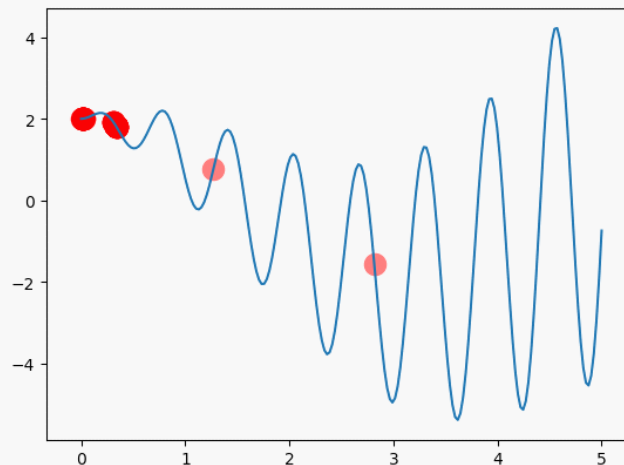
双种群遗传算法示意图

- 双倍体遗传算法
- 双种群遗传算法
- 自适应遗传算法



早熟/停滞：局部收敛

■ Srinivas M., Patnaik L. M.等在1994年提出一种自适应遗传算法(adaptive genetic algorithms, AGA): 交叉概率 P_c 和变异概率 P_m 能随个体的适应度自动改变。



1. 基本思想

■ 对于适应度高于群体平均适应度(f_{avg})的个体, 选择较低的 P_c 和 P_m , 使得该解得以保护进入下一代; 对低于平均适应度(f_{avg})的个体, 选择较高的 P_c 和 P_m , 使该解被淘汰, 进而产生新个体。

交叉个体的适应度

$$P_c = \begin{cases} \frac{k_1(f_{\max} - f')}{f_{\max} - f_{\text{avg}}}, & f' > f_{\text{avg}} \\ k_2, & f' \leq f_{\text{avg}} \end{cases}$$

变异个体的适应度

$$P_m = \begin{cases} \frac{k_3(f_{\max} - f)}{f_{\max} - f_{\text{avg}}}, & f > f_{\text{avg}} \\ k_4, & f \leq f_{\text{avg}} \end{cases}$$

$$k_2 > k_1, \quad k_4 > k_3$$

3. 自适应的交叉概率与变异概率（进一步改进）

■ F—自适应方法：

$$P_c = \begin{cases} P_{c1} - \frac{(P_{c1} - P_{c2})(f' - f_{avg})}{f_{max} - f_{avg}}, & f' > f_{avg} \\ P_{c1}, & f' \leq f_{avg} \end{cases}$$

$$P_m = \begin{cases} P_{m1} - \frac{(P_{m1} - P_{m2})(f - f_{avg})}{f_{max} - f_{avg}}, & f > f_{avg} \\ P_{m1}, & f \leq f_{avg} \end{cases}$$

$$P_{c1} = 0.9, P_{c2} = 0.6, P_{m1} = 0.1, P_{m2} = 0.001$$

当 $f' = f_{max}$, $P_c = P_{c2} > 0$;

当 $f = f_{max}$, $P_m = P_{m2} > 0$ 。

- 最优个体的交叉概率与变异概率最小
- 较优个体要对应于较小的交叉概率与变异概率
- 较差个体对应于最大的交叉概率与变异概率

- 基于**小生境技术**的遗传算法：包括预选机制、排挤机制、共享机制，具有更好的多样性和全局搜索性能。
- **多目标**遗传算法：例如NSGA-II，通过非支配排序和拥挤度比较保持解多样性

没有完美的算法，
只有适合的算法。

“**没有免费的午餐**”定理：
不存在某种最优迭代算
法对所有问题都有效

